Levi9 Summer Practice
Data Engineering Project Documentation

1 Executive Summary

This project implements an end-to-end data pipeline that ingests IMDb's public datasets into a local DuckDB database and uses dbt to transform the raw data into an analysis-ready star schema. The resulting models support questions such as which directors have the highest-rated films, how movie runtimes have changed across decades, and which genres contain the largest number of underrated movies.

2 Dataset Description

2.1 Source and Access

Dataset name	IMDb Non-Commercial Datasets
Source organization	IMDb
Source URL	`https://datasets.imdbws.com/`
Access method	Direct download of IMDb's complete public dataset files
Source format	Compressed, tab-separated values (TSV) files
Pipeline format	Parquet files used as an intermediate, columnar format

2.2 Size and Update Frequency

Files used	Six complete IMDb dataset files
Largest stated tables	`title.basics`: approximately 12.7 million titles; `title.ratings`: approximately 1.7 million rating records
Additional files	`title.crew`, `title.principals`, `name.basics`, and `title.akas`
Extracted size	The complete contents of all six source files; no rows were intentionally filtered out
Date coverage	The full history available in each IMDb weekly dump
Update frequency	Weekly

2.3 Scope and Selection Decisions

The pipeline loads all available records from the six selected IMDb files without applying a date, title-type, language, region, or popularity filter. This broad scope was chosen because the analytical questions require information about titles, ratings, directors and other contributors, genres, alternative titles, and historical trends. Retaining the complete source data also allows the dbt layer to apply different analytical rules without requiring the ingestion process to be changed.

IMDb publishes complete weekly dataset dumps rather than an incremental API for these files. Consequently, each pipeline run performs a full reload instead of processing only new or changed

records. This approach is straightforward and reproducible, although it requires more processing time and local storage than an incremental strategy.

3 Architecture Overview

3.1 System Architecture

The project combines three main tools. Apache Airflow orchestrates the workflow and executes its tasks in the required dependency order. DuckDB provides local analytical storage in a single `dev.duckdb` file. dbt manages the SQL transformation layer, beginning with the preparation of raw data and ending with the final star-schema tables and their data-quality tests.

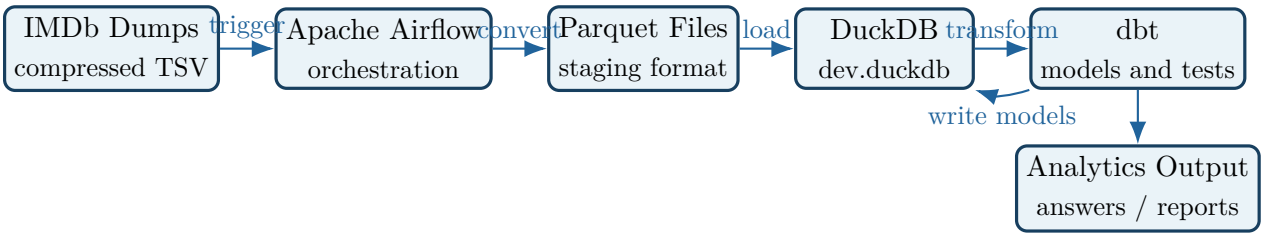


Figure 1: High-level architecture and data flow of the IMDb pipeline.

3.2 Schedule and Trigger Sequence

1. **Pipeline trigger:** The `imdb_ingestion` DAG has no schedule and is started manually from Airflow whenever a new pipeline run is required.
2. **TSV-to-Parquet conversion:** The raw IMDb TSV files are converted into Parquet files, providing an efficient intermediate format for subsequent loading.
3. **DuckDB loading:** The Parquet data is loaded into the local `dev.duckdb` database.
4. **Source validation:** Airflow checks that the loaded source tables are not empty before allowing transformations to continue.
5. **dbt transformation:** Airflow invokes dbt to clean and standardize the raw data and build the final star-schema models.
6. **dbt testing:** The dbt tests validate the generated models and cause the pipeline run to fail if a required data-quality condition is not satisfied.
7. **Completion:** A successful run leaves the transformed, tested analytical tables in `dev.duckdb`, ready to answer the project questions.

The dependency sequence prevents dbt from running against missing or empty source tables. Because the IMDb source is distributed as a complete weekly dump, the workflow uses full-reload semantics:

the source data is regenerated and reloaded on every run before the analytical models are rebuilt and tested.

4 Data Modeling Explanation

The analytical layer follows a star-schema design, with `fct_title_ratings` as the central fact table and several descriptive dimensions surrounding it. Bridge tables resolve the many-to-many relationships between titles, genres, and people. This structure keeps the principal analytical measures in one table while allowing them to be filtered and grouped by descriptive attributes.

4.1 Model Inventory and Grain

Model / table	Entity	Grain	Purpose and rationale
`fct_title_ratings`	Title rating	One row per title	Central fact table containing the title's rating and vote count. It also carries frequently used title attributes, including type, name, release year, and runtime, sourced from `dim_title`.
`dim_title`	Title	One row per title	Stores descriptive title attributes such as name, title type, release year, runtime, and genres.
`dim_person`	Person	One row per person	Stores people who contribute to titles, including actors, directors, and writers.
`dim_genre`	Genre	One row per unique genre	Provides a consistent genre list for grouping titles without repeating genre descriptions.
`dim_year`	Release year	One row per release year	Supports time-based analysis and comparisons across years and decades.
`bridge_title_genres`	Title–genre assignment	One row per title–genre pair	Resolves the many-to-many relationship because one title may have multiple genres and one genre may describe many titles.

Model / table	Entity	Grain	Purpose and rationale
bridge_title_crew	Title–person role	One row per title–person–role combination	Connects titles to people and records whether each person served as a director, writer, actor, or another modeled role.
snp_title_basics	Title metadata version	One row per title per validity period	SCD Type 2 snapshot that preserves changes to title names, genres, and runtimes between weekly IMDb loads.
snp_title_ratings	Title rating version	One row per title per validity period	Snapshot that preserves the history of rating and vote-count changes across pipeline runs.

4.2 Relationships Between Models

Parent model	Child model	Join key	Cardinality / meaning
dim_title	fct_title_ratings	Title identifier	One-to-one at the current model grain; each fact row describes the ratings for one title.
dim_title	bridge_title_genres	Title identifier	One-to-many; one title can appear in multiple title–genre assignments.
dim_genre	bridge_title_genres	Genre identifier	One-to-many; each genre can be assigned to many titles.
dim_title	bridge_title_crew	Title identifier	One-to-many; one title can have multiple credited people and roles.
dim_person	bridge_title_crew	Person identifier	One-to-many; one person can contribute to multiple titles.
dim_year	dim_title	Release year	One-to-many; one calendar year can be associated with many titles.

The model separates entities according to their natural analytical grain. The fact and dimension tables use the title, person, genre, or year identifier appropriate to that entity, while the bridge tables use combinations of foreign keys to represent multivalued relationships without duplicating fact rows. The two snapshots are maintained separately from the current-state star schema because their purpose is historical tracking rather than current-state reporting. Both use SCD Type 2

behavior, creating a new version when a tracked value changes instead of overwriting the previous state.

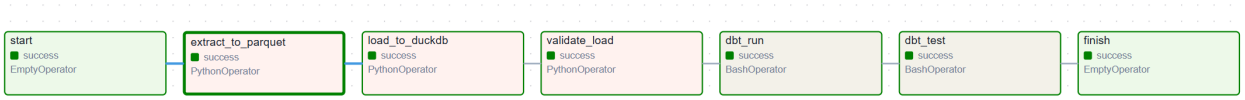
5 Project Structure Walkthrough

```
project-root/
|-- dags/                       # Airflow DAG for the complete pipeline
|-- dbt_project/
|   |-- models/
|   |   |-- staging/            # Source renaming and type casting
|   |   |-- intermediate/       # Reshaping and exploding source fields
|   |   `-- marts/              # Dimensions, bridges, fact, and final queries
|   |-- snapshots/              # SCD Type 2 title and rating history
|   |-- tests/                  # Custom SQL data-quality tests
|   `-- profiles.yml            # dbt profile used inside Docker
|-- raw/                        # Generated Parquet files (git-ignored)
|-- data/                       # Original IMDb TSV.gz files (git-ignored)
`-- docs/                       # Project documentation
```

The folder structure separates orchestration, transformation logic, historical tracking, tests, and generated data. Within dbt, the models are divided into three layers. The `staging` layer contains one model per IMDb source file and performs only column renaming and type conversion. The `intermediate` layer reshapes the staged data, including exploding pipe-separated source columns into individual rows. The `marts` layer contains the final dimensions, bridge tables, fact table, and models used to answer the deliverable questions.

The `snapshots` folder contains the SCD Type 2 definitions for title metadata and rating history, while `tests` contains custom SQL tests for project-specific data-quality rules. These folders, together with the three model layers, are located under `dbt_project`. The original compressed IMDb files are stored in `data`, and the Parquet files generated by Airflow are stored in `raw`. Both data directories are excluded from version control because they contain large source or generated files. The `docs` directory contains the final written documentation.

6 Airflow DAG Explanation



6.1 DAG Configuration

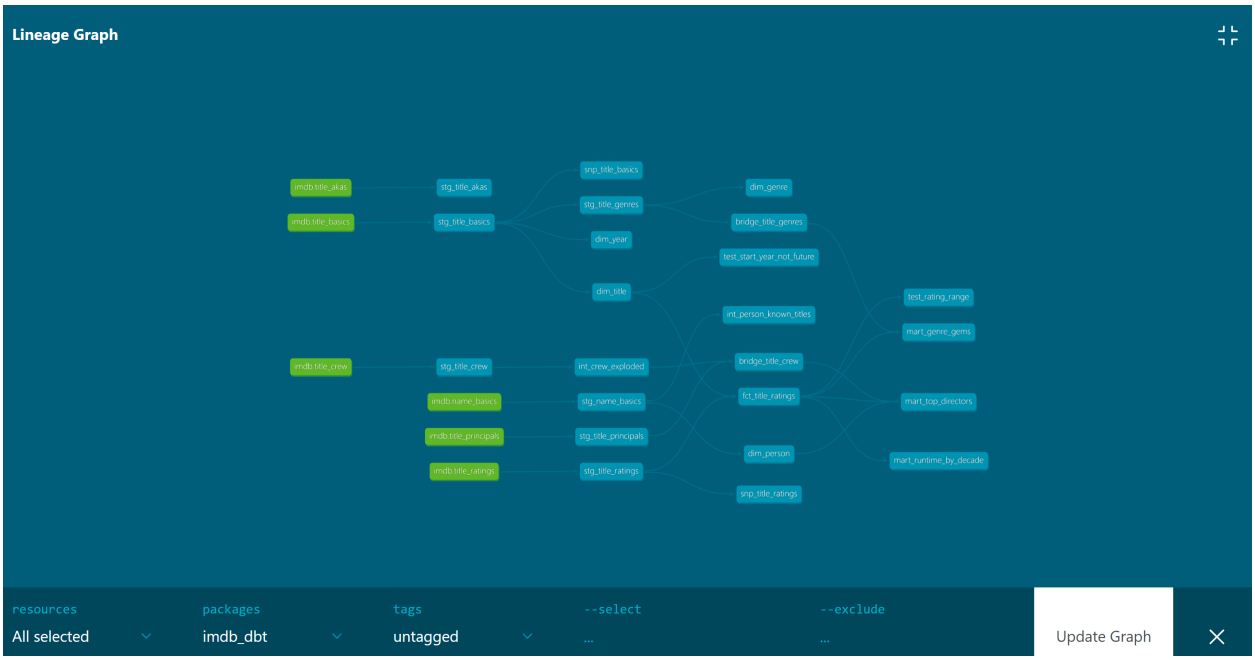
DAG ID	`imdb_ingestion`
Schedule	None; the DAG is not associated with a recurring schedule
Trigger method	Manual trigger from the Airflow UI or command line
Number of tasks	Five processing tasks, plus `start` and `finish` boundary tasks
Execution order	`start` → `extract_to_parquet` → `load_to_duckdb` → `validate_load` → `dbt_run` → `dbt_test` → `finish`

6.2 Task-by-Task Explanation

Task ID	Responsibility	Depends on	Output / success condition
`start`	Marks the beginning of the DAG run using an `EmptyOperator`.	None; first task	The processing chain is allowed to begin.
`extract_to_parquet`	Reads each compressed TSV file from `data` in chunks and writes it as Parquet in `raw`. Chunked processing prevents large files from exhausting memory.	start	One valid Parquet file for every required IMDb source file.
`load_to_duckdb`	Loads each Parquet file into DuckDB using `CREATE OR REPLACE TABLE`. Replacing the tables makes repeated executions idempotent.	extract_to_parquet	All required raw tables exist in `dev.duckdb`.
`validate_load`	Checks that every loaded table contains at least one row.	load_to_duckdb	Validation succeeds for every table; otherwise the pipeline stops.
`dbt_run`	Executes all dbt models in dependency order: staging, intermediate, and marts.	validate_load	All dbt models are built successfully in DuckDB.
`dbt_test`	Runs the complete set of dbt data-quality tests against the generated models.	dbt_run	All critical tests pass; a critical failure marks the pipeline as failed.
`finish`	Marks successful completion of the full DAG using an `EmptyOperator`.	dbt_test	The complete pipeline is shown as successful.

The five processing tasks form a linear dependency chain between the `start` and `finish` boundary tasks because each stage requires the output of the preceding stage. The source files must be converted before they can be loaded, and the loaded tables must pass the non-empty validation before dbt is allowed to build downstream models. Tests run last so that they evaluate the completed analytical layer. If any task fails, Airflow does not execute its downstream tasks, and `finish` is not reached, preventing incomplete or invalid data from being presented as a successful pipeline result.

7 dbt Lineage and Tests



7.1 Lineage Explanation

The lineage graph shows the complete flow from the six raw IMDb sources through the staging, intermediate, and mart layers. Each staging model standardizes one source table, after which the intermediate models reshape multivalued fields and prepare reusable datasets. The mart layer combines these dependencies into dimensions, bridge tables, the central fact table, and the three deliverable models. All 19 dbt models were built successfully during the documented run.

7.2 Implemented Tests

Model	Column / scope	Test	Why the test exists
dim_title	tconst	unique, not_null	Ensures that every title has a present and unique identifier, preserving the dimension's one-row-per-title grain.
dim_person	nconst	unique, not_null	Ensures that every person has a present and unique identifier.
fct_title_ratings	tconst	not_null, relationships	Ensures that every rating is associated with a valid title in dim_title.
fct_title_ratings	start_year	relationships	Ensures that every release year used by the fact table exists in dim_year.
bridge_title_genres	tconst	relationships	Ensures that every title–genre assignment points to a valid title in dim_title.
bridge_title_crew	nconst	relationships (warn)	Checks that crew members exist in dim_person. This is a warning because some IMDb credits do not have matching profiles in name.basics.
fct_title_ratings	average_rating	test_rating_range	Enforces the IMDb rating domain by requiring values to remain between 0 and 10.
dim_title	start_year	test_start_year_not_future (warn)	Flags future release years while allowing announced or planned titles from IMDb to remain in the dataset.

The tests are executed by the final Airflow task, dbt_test, after all models have been built. A test with error severity causes the task and pipeline run to fail. The two documented warning-level conditions are reported for review but do not stop the pipeline because they represent known characteristics of the IMDb source data rather than transformation failures.

8 How to Run the Project

8.1 Prerequisites

- Git, Docker, and Docker Compose installed and running.

- Sufficient free disk space for six complete IMDb datasets, their Parquet copies, and the DuckDB database.
- Network access to download the IMDb files and the packages required during container startup.
- No API key is required because the files are publicly downloadable.

8.2 Clean Reproduction Steps

1. **Clone and enter the project.**

```
git clone [repository-url]
cd [project-directory]
```

2. **Download the IMDb source data.** Download the following six files from `https://datasets.imdbws.com/` and place them in the `data` directory without extracting them:

```
title.basics.tsv.gz
title.ratings.tsv.gz
title.crew.tsv.gz
title.principals.tsv.gz
name.basics.tsv.gz
title.akas.tsv.gz
```

3. **Start from a clean DuckDB file.** Remove the previous local database if it exists. The pipeline will recreate it.

```
rm -f dbt_project/dev.duckdb
```

4. **Start the Docker services.**

```
docker-compose up -d
```

   Allow approximately five minutes for the containers to start and install the required packages.

5. **Open Airflow.** Navigate to `http://localhost:8083` and sign in with username `admin` and password `admin`.

6. **Run the pipeline.** Locate `imdb_ingestion` in the Airflow UI and trigger it manually. Wait until all five tasks are marked successful. A complete run normally takes approximately 10–15 minutes because of the dataset size.

7. **Generate and open the dbt documentation.** After the pipeline succeeds, run:

```
cd dbt_project
dbt docs generate
dbt docs serve --port 8081
```

   Open `http://localhost:8081` to explore the models, tests, column documentation, and lineage graph.

8. **Verify the run.** Confirm that the Airflow graph shows five successful tasks, the dbt run reports 19 successfully built models, and the expected fact, dimension, bridge, snapshot, and deliverable models are visible in dbt docs.

8.3 Expected Result

After a successful clean run, Airflow displays all five tasks in the success state and `dbt_project/dev.duckdb` contains the loaded source tables and transformed analytical models. The final dbt task reports passing tests together with any expected warnings for unmatched crew profiles or future release years. The dbt docs site exposes the complete lineage graph and the three mart models used for the deliverable questions.

9 Answers to the Deliverable Questions

9.1 Question 1: Who are the top 10 directors by average rating?

Model used: `mart_top_directors`

The model filters director records from `bridge_title_crew`, joins them to `fct_title_ratings` and `dim_person`, and groups the results by person. Directors are retained only when they have at least five directed titles and at least 1,000 combined votes. The qualifying directors are then ordered by average rating, and the top ten are returned.

Result:

A-Z primary_name	123 title_co	123 avg_rat	123 total_v
Cat Santarosa	499	9.96	18,726
Christopher Michale Dailey	159	9.88	4,837
Bülent Dogan	139	9.87	2,899
Kubilay Koçak	42	9.87	1,399
Johnathan Ketelhut	27	9.76	1,661
Halil Ibrahim Unal	89	9.73	3,209
T. Suriavelan	22	9.63	4,845
Zhongwei Qiu	32	9.62	1,099
Paul Heising	13	9.58	1,828
James B. Phelps	53	9.56	1,408

9.2 Question 2: Has average movie runtime changed by decade, and does it correlate with rating?

Model used: `mart_runtime_by_decade`

The model groups movies into decades derived from their release year and calculates both average runtime and average rating for each decade. Its output covers the period from the 1950s to the present, allowing the two metrics to be compared over time.

Result:

decade	avg_runtime_minutes	avg_rating	title_count
1,894	45	5.3	1
1,896	61	3.6	1
1,897	100	5.3	1
1,899	135	3.8	1
1,900	59.5	4.5	2
1,903	52.5	5.05	2
1,904	61.5	5.4	2
1,905	65.67	5.1	3
1,906	61	3.83	3
1,907	78	5.1	3
1,908	75.67	3.87	3
1,909	62	3.8	5
1,910	78.36	4.12	11
1,911	59.71	5.07	21
1,912	63.85	5.59	33
1,913	75.71	5.86	77
1,914	90.97	5.68	123
1,915	65.99	5.81	192
1,916	69.02	5.89	260
1,917	70.01	5.84	254
1,918	71.96	6.01	254
1,919	77.22	5.88	280
1,920	75.93	5.95	244
1,921	79.55	5.96	255

123 decade	123 avg_runtime_minutes	123 avg_rating	123 title_count
2,003	96.19	6.16	3,547
2,004	98.07	6.22	3,983
2,005	94.71	6.25	4,356
2,006	96.2	6.19	4,933
2,007	94.43	6.24	5,145
2,008	94.28	6.21	5,859
2,009	92.61	6.24	6,503
2,010	92.65	6.23	6,819
2,011	94.02	6.25	7,340
2,012	98.42	6.25	7,808
2,013	92.18	6.22	8,265
2,014	92.98	6.24	8,831
2,015	93.58	6.19	9,078
2,016	93.29	6.21	9,489
2,017	93.87	6.18	10,017
2,018	94.3	6.1	10,212
2,019	97.65	6.11	10,352
2,020	96.59	6.07	8,323
2,021	93.91	6.12	8,827
2,022	96.5	6.21	10,149
2,023	97.28	6.23	10,377
2,024	97.62	6.22	10,291
2,025	98.96	6.34	9,152
2,026	102.85	6.56	3,478

9.3 Question 3: Which genres have the most hidden gems versus overrated titles?

Model used: `mart_genre_gems`

The model labels a title as a *hidden gem* when its rating is at least 7.5 and it has fewer than 1,000 votes. A title is labeled *overrated* when its rating is below 5.0 and it has at least 10,000 votes. These classifications are aggregated by genre to calculate the number and percentage of titles in each category.

Result:

genre	hidden_gems	overrated	total_titles	hidden_gem_pct
History	24,943	7	50,585	49.31
Documentary	97,836	17	222,154	44.04
Western	7,518	1	17,170	43.79
Family	51,124	60	118,815	43.03
Animation	91,047	27	215,386	42.27
Adventure	82,831	184	197,303	41.98
Reality-TV	40,204	8	96,006	41.88
Biography	14,872	9	35,539	41.85
Game-Show	20,053	3	48,403	41.43
Mystery	32,417	105	80,128	40.46
Music	21,406	18	52,969	40.41
Fantasy	28,442	99	71,377	39.85
War	6,966	3	17,506	39.79
Comedy	207,162	322	523,556	39.57
Crime	71,596	117	181,614	39.42

10 Challenges and Lessons Learned

10.1 Challenge 1: Invalid Values in `runtimeMinutes`

Problem and symptoms The staging model failed while converting `runtimeMinutes` to an integer. Some records contained non-numeric values, including strings such as `Reality-TV`, even though the column is expected to contain a duration.

Root cause The IMDb source contains dirty values that do not conform to the documented numeric type, and a direct `::INTEGER` cast aborts when it encounters such a value.

Resolution The transformation was changed to `TRY_CAST(runtimeMinutes AS INTEGER)`, which converts valid values and returns `NULL` for invalid ones instead of stopping the pipeline.

Lesson learned External source schemas should be treated as expectations rather than guarantees. Defensive casts allow invalid records to be isolated and tested without making the entire load unusable.

10.2 Challenge 2: Inconsistent Identifier Names Between Models

Problem and symptoms A downstream model referenced the wrong title identifier and failed only when it was executed against the complete dataset.

Root cause `stg_title_akas` renames `titleId` to `title_id`, whereas `stg_title_basics` retains the IMDb identifier name `tconst`. The initial `stg_`

`title_genres` logic assumed that both staging models used the same column name.

Resolution The downstream reference was corrected to use the actual column exposed by `stg_title_basics`.

Lesson learned Staging-layer naming conventions should be standardized and verified through model documentation or contracts before downstream SQL is written.

10.3 Challenge 3: dbt Profile Paths in Local and Docker Environments

Problem and symptoms A dbt configuration that connected successfully in one environment could not locate `dev.duckdb` in the other environment.

Root cause Local dbt execution requires a path relative to the developer's working directory, while the Docker container resolves files from a different filesystem location and therefore requires its own path configuration.

Resolution Two separate profile files were maintained: a local profile in `~/.dbt/profiles.yml` and a container-specific profile inside `dbt_project`.

Lesson learned Environment-dependent paths should be isolated in environment-specific configuration instead of being embedded in transformation models.